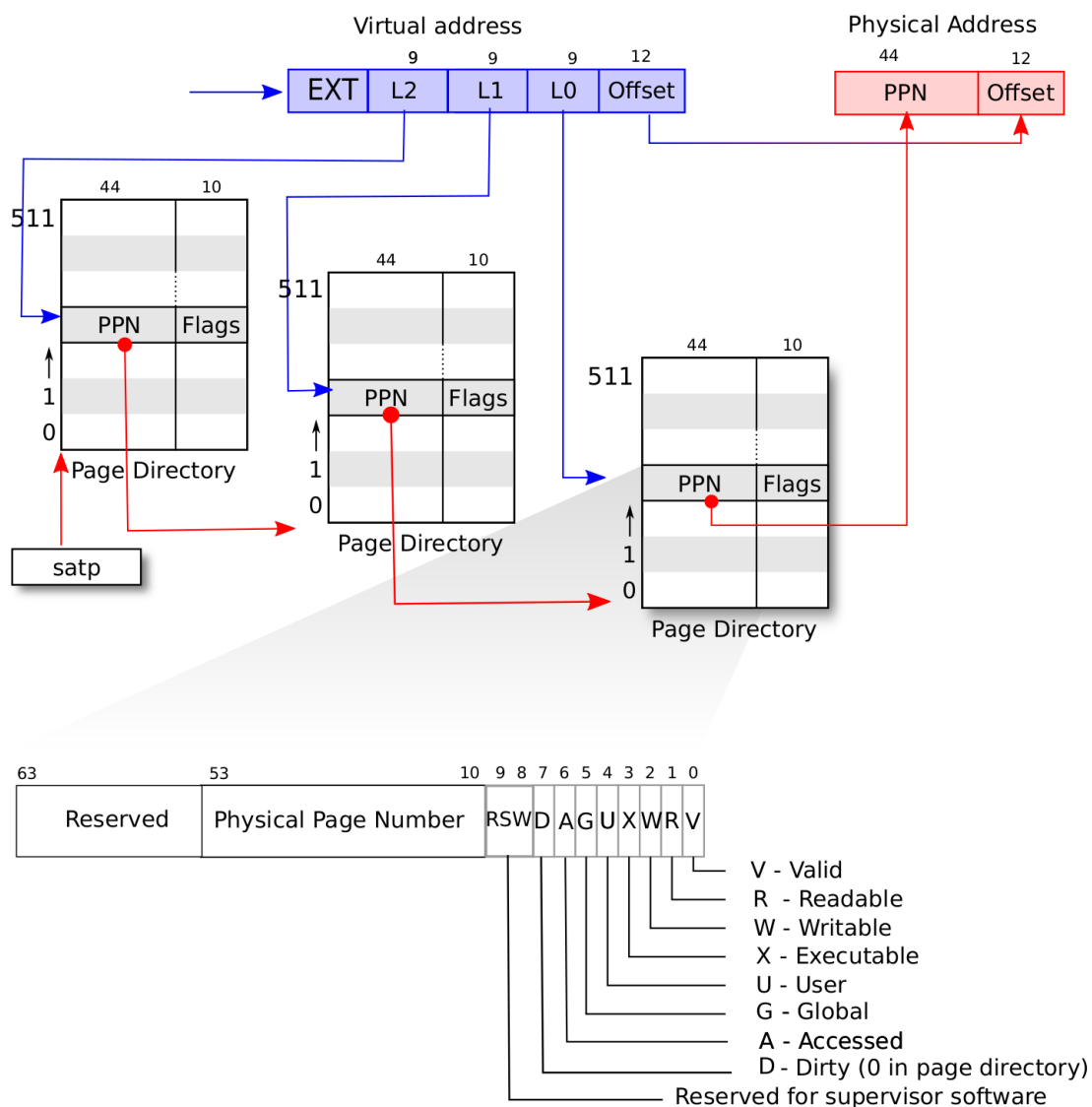
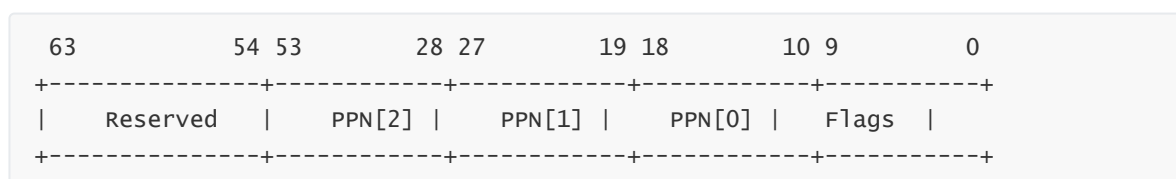


内存页表



页表项结构(RISC-V Sv39):



名词

- 页表 `typedef uint64 *pagetable_t`: 地址转换的核心任务在于如何维护虚拟页号到物理页号的映射。用于构建多级页表结构。
 - 是一个指针,指向512个连续的PTE数组。一个页表占用一个4KB页面。
- 页表项 (PTE, Page Table Entry) `typedef uint64 pte_t`: 物理页号和全部的标志位以某种固定的格式保存在一个结构体中, 是利用虚拟页号在页表中查到的结果。
 - 存储单个页面 (4k) 映射信息
 - 包含物理页号(PPN)和权限标志位
- 页目录项(Page Directory Entry) `typedef uint64 pde_t`

- 本质上与PTE相同，指向下一级页表的物理地址，在RISC-V中PDE和PTE使用相同的格式。

页表目录 (L2) -> 页表 (L1) -> 页表项 (L0) -> 物理页面 (Physical Page)

变量

- trampoline：用户的虚拟地址起始位置

函数

- bin_loader：为每个进程的pg分配一张页表的空间，分配好了进程的页表后再把trap (4k) 和 ustack (4k) 映射 (mappages) 到页表pg。
- 页表实现va->pa的转换过程
 - vm.c 的 walk 模拟CPU进行MMU的过程。输入**页表和待转换的虚拟地址**，通过SV39的转换返回了 页表项 (PTE, Page Table Entry)。
 - vm.c 的 walk_addr 函数根据用户的虚拟地址和页表项，找到对应的物理地址。它不包含虚拟地址里用到的偏置项，如果需要偏置优先采用 user_addr。
- 页表的建立过程
- 启用页表后的跨页表操作：用户态和内核态的内存拷贝

问答作业

请列举 SV39 页表页表项的组成，结合课堂内容，描述其中的标志位有何作用 / 潜在作用？

Sv39 页表页表项的组成如下：

1. **有效位 (V)**：这是页表项的最高位，用于指示页表项是否有效。如果有效位设置为 1，表示页表项有效，可以使用；如果设置为 0，表示页表项无效，禁止使用。这是虚拟内存中页表项的基本有效性标志。
2. **写入位 (W)**：这个标志位用于指示是否可以对此页进行写入操作。如果设置为 1，表示允许写入；如果设置为 0，表示禁止写入。它是页表项的访问权限控制标志之一。
3. **用户位 (U)**：用户位用于指示是否允许用户态程序访问此页。如果设置为 1，表示允许用户态访问；如果设置为 0，表示只允许内核态访问。它是页表项的访问权限控制标志之一。
4. **执行位 (X)**：执行位用于指示是否允许执行此页上的指令。如果设置为 1，表示允许执行；如果设置为 0，表示禁止执行。它也是页表项的访问权限控制标志之一。
5. **全局位 (G)**：全局位用于指示此页是否是全局的，即无需 TLB 缓存，通常用于内核页。如果设置为 1，表示是全局的；如果设置为 0，表示不是全局的。
6. **已访问位 (A)**：已访问位表示是否已经访问过此页，通常由硬件设置。操作系统可以用它来实现页面置换算法，如 LRU。
7. **已修改位 (D)**：已修改位表示是否已经对此页进行了写入操作。与已访问位类似，操作系统可以用它来实现页面置换算法。
8. **物理页框地址 (PPN)**：这是页表项中存储的物理页框的地址。它指示了虚拟页到物理页的映射关系。

Sv39 页表的页表项标志位允许操作系统和硬件实现对虚拟内存的细粒度控制和保护。不同的标志位组合可以实现不同级别的内存保护和权限控制，从而提高系统的安全性和可用性。例如，有效位、写入位、用户位和执行位的不同组合可以实现不同级别的内存保护，使操作系统可以将不同的内存区域分配给用户态和内核态，并设置不同的权限。已访问位和已修改位则用于实现页面置换算法，帮助操作系统

决定哪些页面应该被置换出去，以优化内存利用率。全局位可以用于标识全局共享的页，从而节省 TLB 缓存空间。物理页框地址是页表项的核心，它建立了虚拟地址到物理地址的映射关系，使虚拟内存管理成为可能。

缺页

- 缺页指的是进程访问页面时页面不在页表中或在页表中无效的现象，此时 MMU 将会返回一个中断，告知 os 进程内存访问出了问题。os 选择填补页表并重新执行异常指令或者杀死进程。

- 请问哪些异常可能是缺页导致的？

缺页异常是由于进程访问的页面不在页表中或者在页表中无效而引发的异常。以下这些异常可能是因为缺页导致的：

- Load Page Fault (Load 异常)：当进程试图读取一个不在页表中或者无效的页面时，会引发 Load Page Fault 异常。在 RISC-V 中，这个异常对应的异常代码是 5。
- Store Page Fault (Store 异常)：当进程试图写入一个不在页表中或者无效的页面时，会引发 Store Page Fault 异常。在 RISC-V 中，这个异常对应的异常代码是 7。
- Instruction Page Fault (指令页异常)：当进程试图执行一个不在页表中或者无效的页面上的指令时，会引发 Instruction Page Fault 异常。在 RISC-V 中，这个异常对应的异常代码是 12。

```
enum Exception {
    InstructionMisaligned = 0,
    InstructionAccessFault = 1,
    IllegalInstruction = 2,
    Breakpoint = 3,
    LoadMisaligned = 4,
    LoadAccessFault = 5,
    StoreMisaligned = 6,
    StoreAccessFault = 7,
    UserEnvCall = 8,
    SupervisorEnvCall = 9,
    MachineEnvCall = 11,
    **InstructionPageFault = 12,
    LoadPageFault = 13,
    StorePageFault = 15,**
};
```

- 发生缺页时，描述相关的重要寄存器的值 (lab2 中描述过的可以简单点)。
- `sepc` (Exception Program Counter)：trap 发生时会将当前指令的下一条指令地址写入其中，用于 trap 处理完成后返回。
- `stval` (Machine Trap Value)：mtval 寄存器包含导致异常的原因，即导致异常的指令的具体信息。例如，如果是缺页异常，那么 mtval 寄存器包含导致缺页异常的虚拟地址。
- `scause`：中断/异常发生时，CSR 寄存器 scause 中会记录其信息，Interrupt 位记录是中断还是异常，Exception Code 记录中断/异常的种类。
- `sstatus`：记录处理器当前状态，其中 SPP 段记录当前特权等级。
- `stvec`：记录处理 trap 的入口地址，现有两种模式 Direct 和 Vectored。
- `sscratch`：其中的值是指向 hart 相关的 S 态上下文的指针，比如内核栈的指针。

- 缺页有两个常见的原因，其一是 Lazy 策略，也就是直到内存页面被访问才实际进行页表操作。比如，一个程序被执行时，进程的代码段理论上需要从磁盘加载到内存。但是 os 并不会马上这样做，而是会保存 .text 段在磁盘的位置信息，在这些代码第一次被执行时才完成从磁盘的加载操作。

- 这样做有哪些好处？

缺页有两个常见的原因，其一是 **Lazy 策略**，也就是直到内存页面被访问才实际进行页表操作。比如，一个程序被执行时，进程的代码段理论上需要从磁盘加载到内存。但是 os 并不会马上这样做，而是会保存 **.text 段在磁盘的位置信息**，在这些代码第一次被执行时才完成从磁盘的加载操作。

Lazy Loading 策略有以下好处：

1. **减少初始化开销**：Lazy Loading 允许操作系统在程序启动时只加载必需的页面，而不是一次性加载整个程序。这可以减少启动时间和初始化开销，因为不需要将整个程序加载到内存中。
 2. **节省内存**：Lazy Loading 策略避免了不必要的内存占用。如果程序的某些部分从不被访问，那么它们就不会被加载到内存中，从而节省了内存资源。
 3. **提高响应速度**：通过仅在需要时加载页面，Lazy Loading 可以提高系统的响应速度。只有当程序访问某个页面时，操作系统才会执行磁盘加载操作，而不会在程序启动时浪费时间加载可能永远不会被访问的内容。
 4. **更好的磁盘利用率**：Lazy Loading 允许操作系统将程序的不同部分分散在磁盘上，根据需要加载。这可以提高磁盘利用率，因为不需要在磁盘上为整个程序分配连续的空间。
- 此外 **COW(Copy On Write)** 也是常见的容易导致缺页的 **Lazy 策略**，这个之后再讲。其实，我们的 `mmap` 也可以采取 Lazy 策略，比如：一个用户进程先后申请了 10G 的内存空间，然后用了其中 1M 就直接退出了。按照现在的做法，我们显然亏大了，进行了很多没有意义的页表操作。

- 请问处理 10G 连续的内存页面，需要操作的页表实际大致占用多少内存（给出数量级即可）？

处理 10GB 连续的内存页面所需的页表实际上占用的内存量取决于操作系统的页表结构和管理策略。在 RISC-V 的页表结构中，一个**页表项 (Page Table Entry, PTE)** 通常占据 8 字节（64 位系统），其中包括物理页框号和一些标志位。让我们假设一个 PTE 占用 8 字节。

为了估算 10GB 连续内存页面所需的页表实际占用内存量，我们可以按照以下步骤进行计算：

1. 首先，将 10GB 转换为字节数。1GB 等于 1,073,741,824 字节，所以 **10GB** 等于 $10 * 1,073,741,824 = 10,737,418,240$ 字节。
2. 然后，计算每个页面表项覆盖的内存范围。假设每个页面表项管理 4KB ($4 * 1024$ 字节) 的内存页面。
3. 计算需要多少个页面表项来管理 10GB 的内存。这可以通过将 10GB 除以每个页面表项管理的内存范围来实现。
4. 最后，将所需的页面表项数量乘以每个 PTE 的大小来估算所需的总内存量。

让我们进行具体计算：

- 内存大小：10,737,418,240 字节
- 每个页面表项管理的内存范围：4KB = $4 * 1024$ 字节
- 需要的页面表项数量：10,737,418,240 字节 / 4KB = 2,621,440 个页表项

假设每个页表项占用 8 字节，则需要的总内存量为：

$2,621,440 \text{ 个页表项} * 8 \text{ 字节/页表项} = 20,971,520 \text{ 字节}$

所以，处理 10GB 连续的内存页面所需的页表实际占用内存量约为 20,971,520 字节，或者大约 20MB。这只是一个估算，实际内存占用可能会因操作系统的管理策略和对齐等因素而有所不同。

- 请简单思考如何才能现有框架基础上实现 Lazy 策略，缺页时又该如何处理？描述合理即可，不需要考虑实现。

要在现有框架基础上实现 Lazy 策略，可以采取以下简单思路：

1. **延迟加载 (Lazy Loading)**：在用户进程请求内存映射时，不立即将整个内存区域加载到物理内存中。而是仅创建虚拟内存映射和页表项，记录对应的磁盘位置等信息。
 2. **缺页处理 (Page Fault Handling)**：当用户进程访问虚拟内存中的某个尚未加载的内存页面时，会触发缺页异常。在缺页异常处理程序中，操作系统会根据页表中的磁盘位置信息，将相应的磁盘数据加载到物理内存中，并更新页表项，使其指向新加载的物理页面。
 3. **惰性加载 (Demand Paging)**：为了提高性能，可以采用惰性加载策略，即只加载实际被访问的内存页面，而不是一次性加载整个区域。这可以通过在缺页处理程序中进行懒加载操作来实现。
 4. **内存回收 (Memory Reclamation)**：当系统内存不足时，操作系统可以选择回收一些不常访问的内存页面，将其写回磁盘，并更新页表项为无效。这需要根据页面访问模式和策略来确定哪些页面可以被回收。
 5. **性能优化**：为了提高性能，可以采用预读取 (Prefetching) 策略，即在缺页处理时，不仅加载当前访问的页面，还预先加载相邻的页面，以减少未来可能的缺页次数。
- 缺页的另一个常见原因是 swap 策略，也就是内存页面可能被换到磁盘上了，导致对应页面失效。

- 此时页面失效如何表现在页表项(PTE)上？

Dirty bit (D 位)：当页面被修改并且尚未写回到主存时，该位会被设置为 1。如果页面已经被换出到磁盘上，D 位将保持为 1，以指示页面数据已过期。

Valid bit (V 位)：当页面在主存中有效时，V 位被设置为 1。如果页面被换出到磁盘上，V 位将被清除为 0，表示该页无效。

通过检查页表项的 D 位和 V 位，操作系统可以确定页面是否需要从磁盘重新加载到内存中。如果 D 位为 1，说明页面需要写回到主存，在将其置为有效之前，必须将页数据从磁盘读取到内存中。如果 V 位为 0，说明页面当前无效，需要将其从磁盘加载到内存中，并将 V 位设置为 1，表示页面有效。

- 双页表与单页表

为了防范侧信道攻击，我们的 os 使用了双页表。但是传统的设计一直是单页表的，也就是说，用户线程和对应的内核线程共用同一张页表，只不过内核对应的地址只允许在内核态访问。请结合课堂知识回答如下问题：（备注：这里的单/双的说法仅为自创的通俗说法，并无这个名词概念，详情见 [KPTI](#)）

- 单页表情况下，如何更换页表？

在单页表情况下，页表的更换通常是由操作系统的上下文切换来触发的。当从用户态切换到内核态或从一个进程切换到另一个进程时，操作系统会根据相应的上下文信息加载不同的页表，实现页表的更换。

- 单页表情况下，如何控制用户态无法访问内核页面？（tips:看看第一题最后一问）

- 设置页面权限：内核页面通常会被设置为只能在内核态下访问（例如，设置 PTE_U 位为 0），这样用户态无法访问内核页面。
- 操作系统权限：操作系统内核态拥有较高的权限，可以通过特权级别或访问控制机制来确保用户态无法直接访问内核页面。用户程序只能通过系统调用进入内核态，并在内核态下由操作系统执行，从而实现对内核页面的访问控制。

- 单页表有何优势？（回答合理即可）

单页表的主要优势在于简化了地址转换过程，减少了内存访问的开销。由于用户线程和内核线程共享同一张页表，不需要在上下文切换时频繁切换页表，这可以提高地址转换的效率。此外，单页表还可以节省内存，因为不需要为每个用户线程分配独立的页表。

- 双页表实现下，何时需要更换页表？假设你写一个单页表操作系统，你会选择何时更换页表（回答合理即可）？

在双页表实现下，页表的更换通常在发生上下文切换时需要。当从用户态切换到内核态或从一个进程切换到另一个进程时，需要加载相应的页表，以确保正确的地址转换。如果操作系统采用了每个进程独立的页表，那么在进程切换时需要更换页表。

如果我写一个单页表操作系统，我会选择在发生进程切换时更换页表，因为这是最频繁的上下文切换情况之一。在其他情况下，如从用户态切换到内核态，可能不需要更换整张页表，而只需修改页表项的权限位来实现访问控制。这样可以减少页表更换的开销，提高性能。